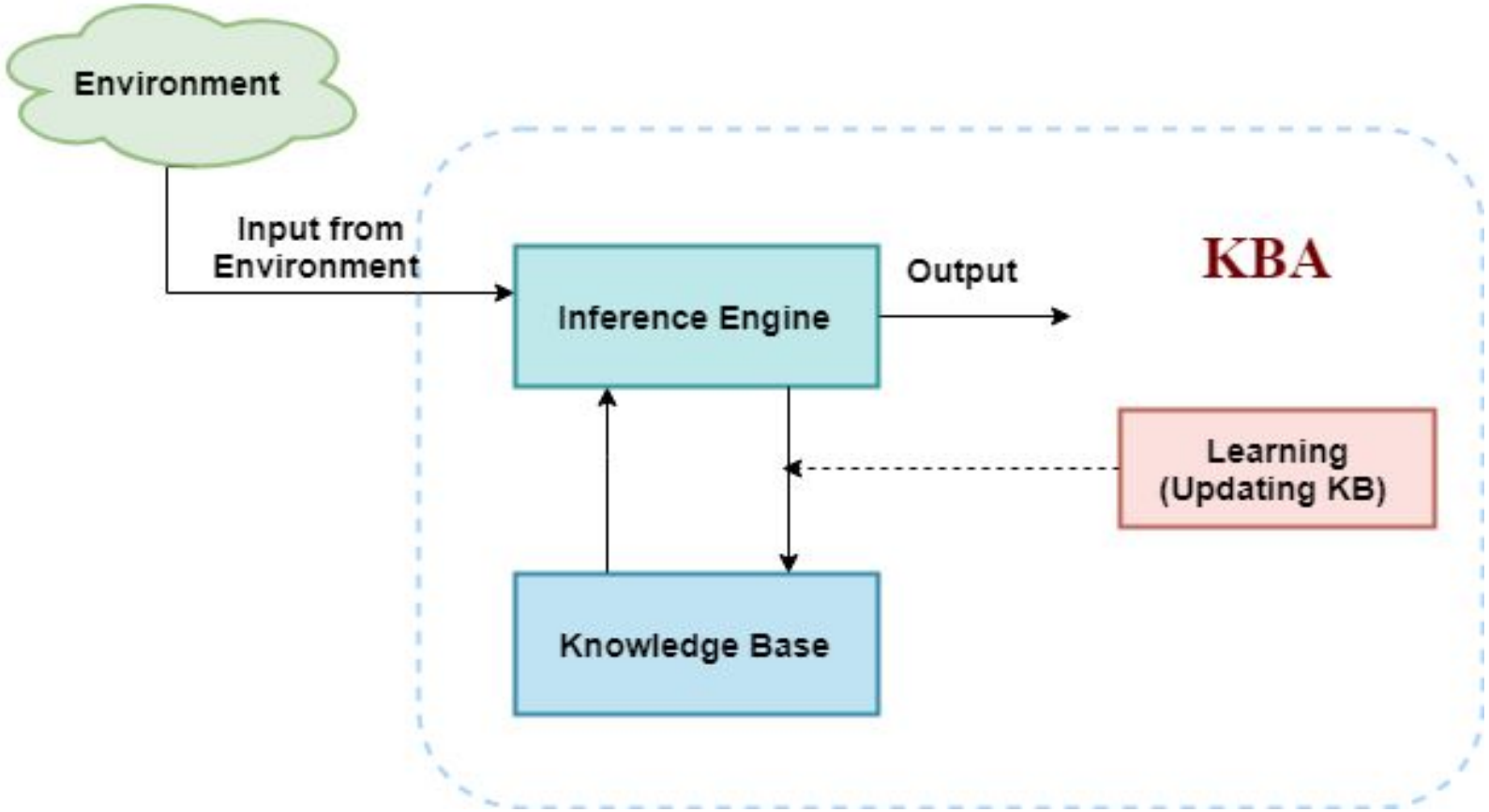
UNIT-3

KNOWLEDGE REPRESENTATION

3.1 Knowledge-Based Agents

- An intelligent agent needs knowledge about the real world for taking decisions and reasoning to act efficiently
- Knowledge-based agents are those agents who have the capability of maintaining an internal state of knowledge, reason over that knowledge, update their knowledge after observations and take actions.
- These agents can represent the world with some formal representation and act intelligently.

- Knowledge-based agents are composed of two main parts:
  - Knowledge-base and
  - Inference system.



This diagram is representing a generalized architecture for a knowledge-based agent. The knowledge-based agent (KBA) take input from the environment by perceiving the environment. The input is taken by the inference engine of the agent and which also communicate with KB to decide as per the knowledge store in KB. The learning element of KBA regularly updates the KB by learning new knowledge

Knowledge base: Knowledge-base is a central component of a knowledge-based agent, it is also known as KB. It is a collection of sentences (here 'sentence' is a technical term and it is not identical to sentence in English). These sentences are expressed in a language which is called a knowledge representation language. The Knowledge-base of KBA stores fact about the world.

Why use a knowledge base?

Knowledge-base is required for updating knowledge for an agent to learn with experiences and take action as per the knowledge.

Inference system

Inference means deriving new sentences from old. Inference system allows us to add a new sentence to the knowledge base. A sentence is a proposition about the world. Inference system applies logical rules to the KB to deduce new information.

Inference system generates new facts so that an agent can update the KB. An inference system works mainly in two rules which are given as:

- Forward chaining
- Backward chaining

Operations Performed by KBA

Following are three operations which are performed by KBA in order to show the intelligent behavior:

1. **TELL:** This operation tells the knowledge base what it perceives from the environment.
2. **ASK:** This operation asks the knowledge base what action it should perform.
3. **Perform:** It performs the selected action.

A generic knowledge-based agent:

Following is the structure outline of a generic knowledge-based agents program:

```
function KB-AGENT(percept) returns an action
  persistent: KB, a knowledge base
              t, a counter, initially 0, indicating time

  TELL(KB, MAKE-PERCEPT-SENTENCE(percept, t))
  action ← ASK(KB, MAKE-ACTION-QUERY(t))
  TELL(KB, MAKE-ACTION-SENTENCE(action, t))
  t ← t + 1
  return action
```

Figure 7.1 A generic knowledge-based agent. Given a percept, the agent adds the percept to its knowledge base, asks the knowledge base for the best action, and tells the knowledge base that it has in fact taken that action.

The knowledge-based agent takes percept as input and returns an action as output. The agent maintains the knowledge base, KB, and it initially has some background knowledge of the real world. It also has a counter to indicate the time for the whole process, and this counter is initialized with zero.

Each time when the function is called, it performs its three operations:

- Firstly it TELLs the KB what it perceives.
- Secondly, it asks KB what action it should take
- Third agent program TELLS the KB that which action was chosen.

The **MAKE-PERCEPT-SENTENCE** generates a sentence as setting that the agent perceived the given percept at the given time.

The **MAKE-ACTION-QUERY** generates a sentence to ask which action should be done at the current time.

The **MAKE-ACTION-SENTENCE** generates a sentence which asserts that the chosen action was executed.

Various levels of knowledge-based agent:

A knowledge-based agent can be viewed at different levels which are given below:

1. Knowledge level

Knowledge level is the first level of knowledge-based agent, and in this level, we need to specify what the agent knows, and what the agent goals are. With these specifications, we can fix its behavior. For example, suppose an automated taxi agent needs to go from a station A to station B, and he knows the way from A to B, so this comes at the knowledge level.

2. Logical level:

At this level, we understand that how the knowledge representation of knowledge is stored. At this level, sentences are encoded into different logics. At the logical level, an encoding of knowledge into logical sentences occurs. At the logical level we can expect to the automated taxi agent to reach to the destination B.

3. **Implementation level:**

This is the physical representation of logic and knowledge. At the implementation level agent perform actions as per logical and knowledge level. At this level, an automated taxi agent actually implement his knowledge and logic so that he can reach to the destination.

Approaches to designing a knowledge-based agent:

There are mainly two approaches to build a knowledge-based agent:

1. Declarative approach: We can create a knowledge-based agent by initializing with an empty knowledge base and telling the agent all the sentences with which we want to start with. This approach is called Declarative approach.

2. Procedural approach: In the procedural approach, we directly encode desired behavior as a program code. Which means we just need to write a program that already encodes the desired behavior or agent.

However, in the real world, a successful agent can be built by combining both, declarative and procedural approaches, and declarative knowledge can often be compiled into more efficient procedural code.

Wumpus World

- The wumpus world is a cave consisting of rooms connected by passageways. Lurking somewhere in the cave is the terrible wumpus, a beast that eats anyone who enters its room.
- The wumpus can be shot by an agent, but the agent has only one arrow. Some rooms contain bottomless pits that will trap anyone who wanders into these rooms (except for the wumpus, which is too big to fall in). The only mitigating feature of this bleak environment is the possibility of finding a heap of gold.

PEAS Environment for Wumpus World

Performance Measure:

+1000 for exiting with gold, −1000 for dying, −1 per action, −10 per arrow use.

Environment:

4×4 grid, start at [1,1] facing right. Gold & Wumpus randomly placed (not at start).
Each non-start square has 20% chance of a pit.

Actuators:

Move Forward, Turn Left/Right, Grab gold, Shoot arrow (once), Climb (only at [1,1]).
Death if entering pit or live Wumpus.

Sensors:

Stench near Wumpus.
Breeze near pit.
Glitter on gold square.
Bump if hit wall.
Scream when Wumpus dies.

For example, if there is a stench and a breeze, but no glitter, bump, or scream, the agent program will get [Stench, Breeze, None, None, None].

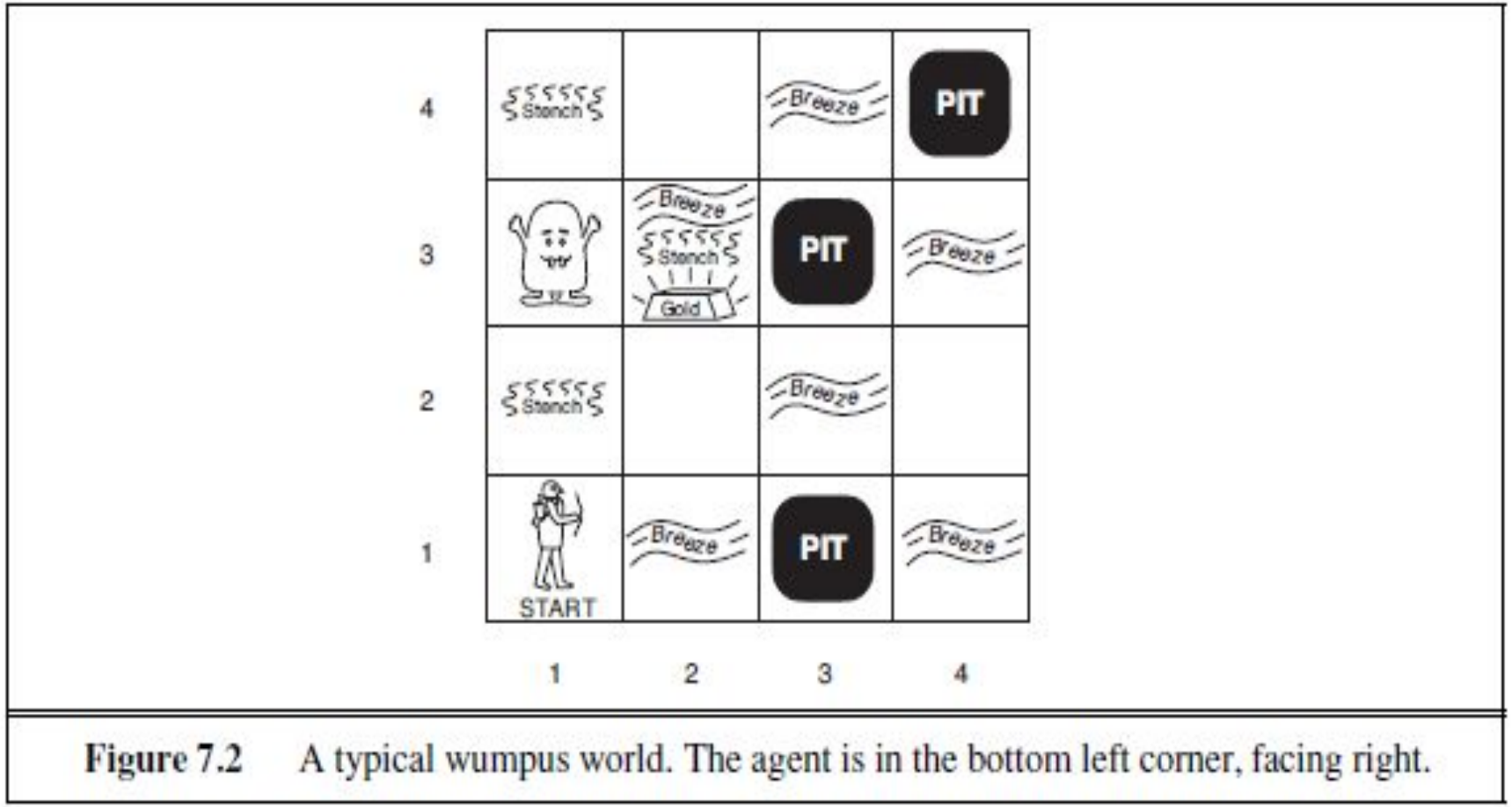


Figure 7.2 A typical wumpus world. The agent is in the bottom left corner, facing right.

How it works:

- The grid is a 4×4 world.
- The agent starts at the bottom-left corner (1,1), facing right.

- There are hazards:
  - PIT: deadly pit (if agent steps in, it dies).
  - Wumpus: a monster that kills the agent if they enter its cell.
- There is Gold: the agent's goal is to find and grab it, then exit.

Percepts (Clues):

  - Stench: in any square adjacent to the Wumpus.
  - Breeze: in any square adjacent to a pit.
  - Gold glitter: in the square containing the gold.

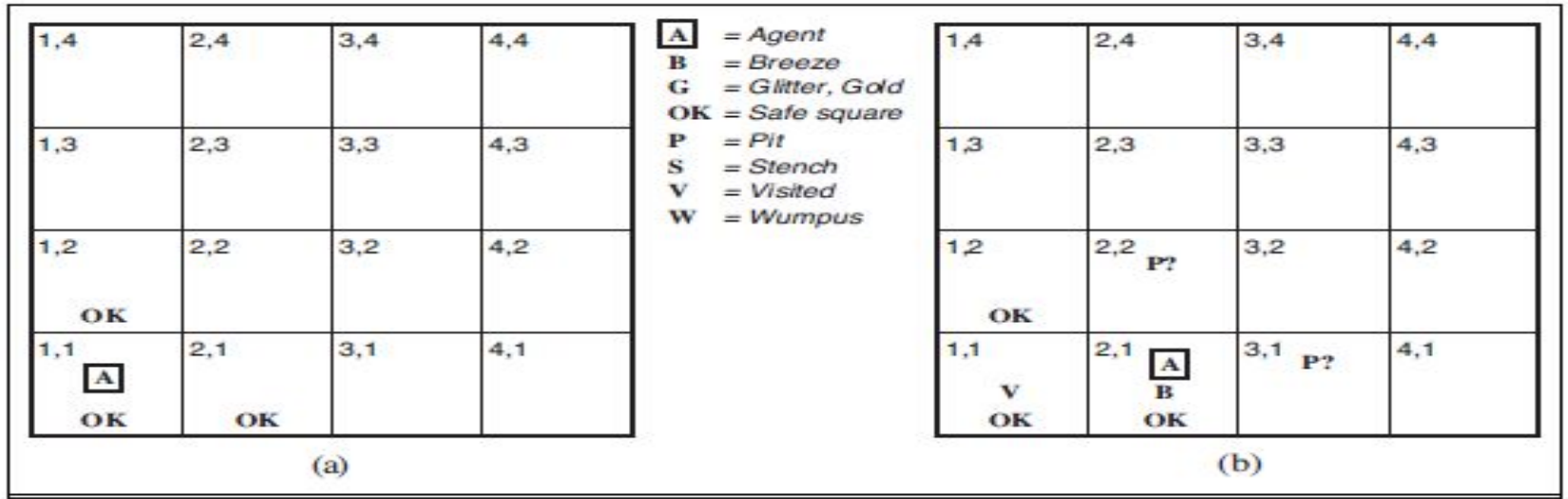


Figure 7.3 The first step taken by the agent in the wumpus world. (a) The initial situation, after percept [*None*, *None*, *None*, *None*, *None*]. (b) After one move, with percept [*None*, *Breeze*, *None*, *None*, *None*].

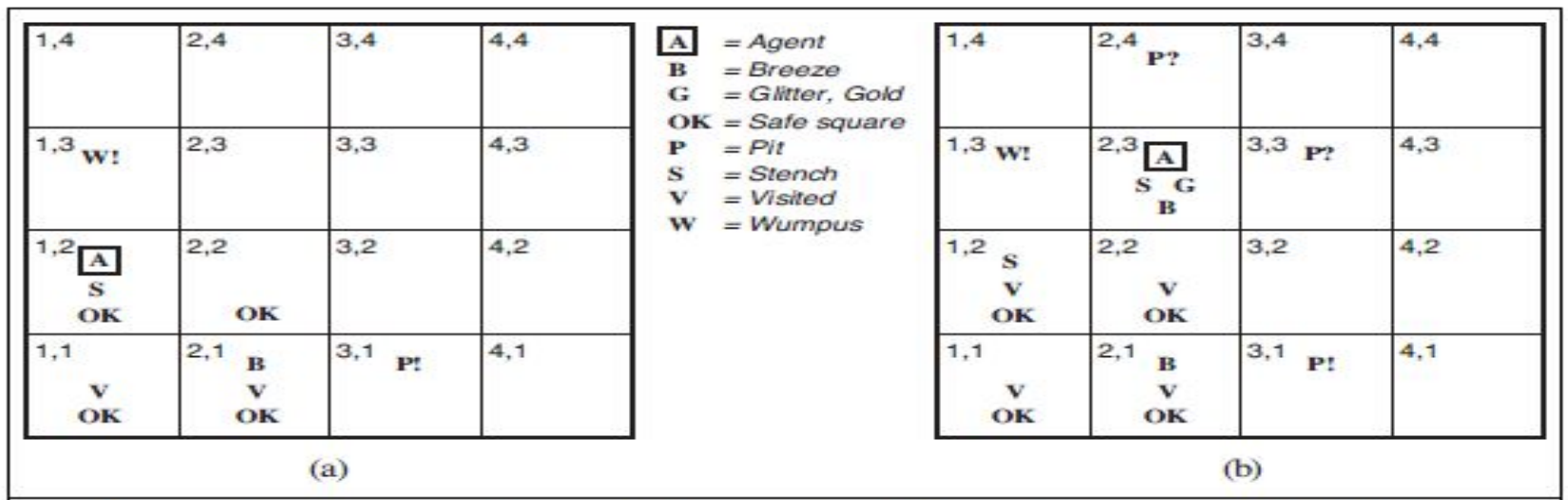


Figure 7.4 Two later stages in the progress of the agent. (a) After the third move, with percept [*Stench*, *None*, *None*, *None*, *None*]. (b) After the fifth move, with percept [*Stench*, *Breeze*, *Glitter*, *None*, *None*].

- Start: Agent at [1,1], knows it's safe (A, OK).Percept: [None, None, None, None, None] → Neighboring squares [1,2] and [2,1] are safe.
- Move to [2,1]:
  - Percept: Breeze (B) → Pit nearby in [2,2] or [3,1] (marked P?).
  - Only other safe unvisited square is [1,2]. Agent returns to [1,1] and moves to [1,2].
- At [1,2]:
  - Percept: Stench (S) → Wumpus nearby.
  - Cannot be in [1,1] or [2,2] → Wumpus is in [1,3] (W!).
  - No breeze here → no pit in [2,2].
  - Earlier info said pit is in [2,2] or [3,1] → must be in [3,1].
- Inference: [2,2] is safe → Agent can move there, then to [2,3].
- At [2,3]:Percept: Glitter → Gold found! Grab it and return to start.

3.2 Logic

- These sentences are expressed according to the syntax of the representation language, which specifies all the sentences that are well formed.
- The notion of syntax is clear enough in ordinary arithmetic: “x + y = 4” is a well-formed sentence, whereas “x4y+ =” is not.
- logic must also define the semantics or meaning of sentences. The semantics defines the truth of each sentence with respect to each possible world.
- For example, the semantics for arithmetic specifies that the sentence “x + y =4” is true in a world where x is 2 and y is 2, but false in a world where x is 1 and y is 1. In standard logics, every sentence must be either true or false in each possible world—there is no “in between.”

- When being precise, we use the term model instead of “possible world.”
    - Possible world: Think of it as a real situation the agent could be in.
    - Model: A mathematical description that decides whether each statement is true or false.
- Example: If a possible world has x men and y women at a table,
    - The statement x + y = 4 is true if there are exactly 4 people.
    - Formally, models are all possible assignments of numbers to x and y.
- A model m satisfies a sentence α if α is true in that model.
- We write M(α) to mean “the set of all models where α is true.”

- Entailment means that one sentence logically follows from another.
- In math notation:

$$\alpha \models \beta$$

- This means: If α is true, then β must also be true.
- Sentence α entails sentence β if, in every model where α is true, β is also true.
- Formally:

$$\alpha \models \beta \text{ if and only if } M(\alpha) \subseteq M(\beta)$$

This means: all models that make α true also make β true.

α is a stronger statement than β because it rules out more possibilities.

Example (arithmetic):

Example (arithmetic):
- α: `x = 0`
- β: `x * y = 0`

If x = 0, then x * y = 0 no matter what y is.

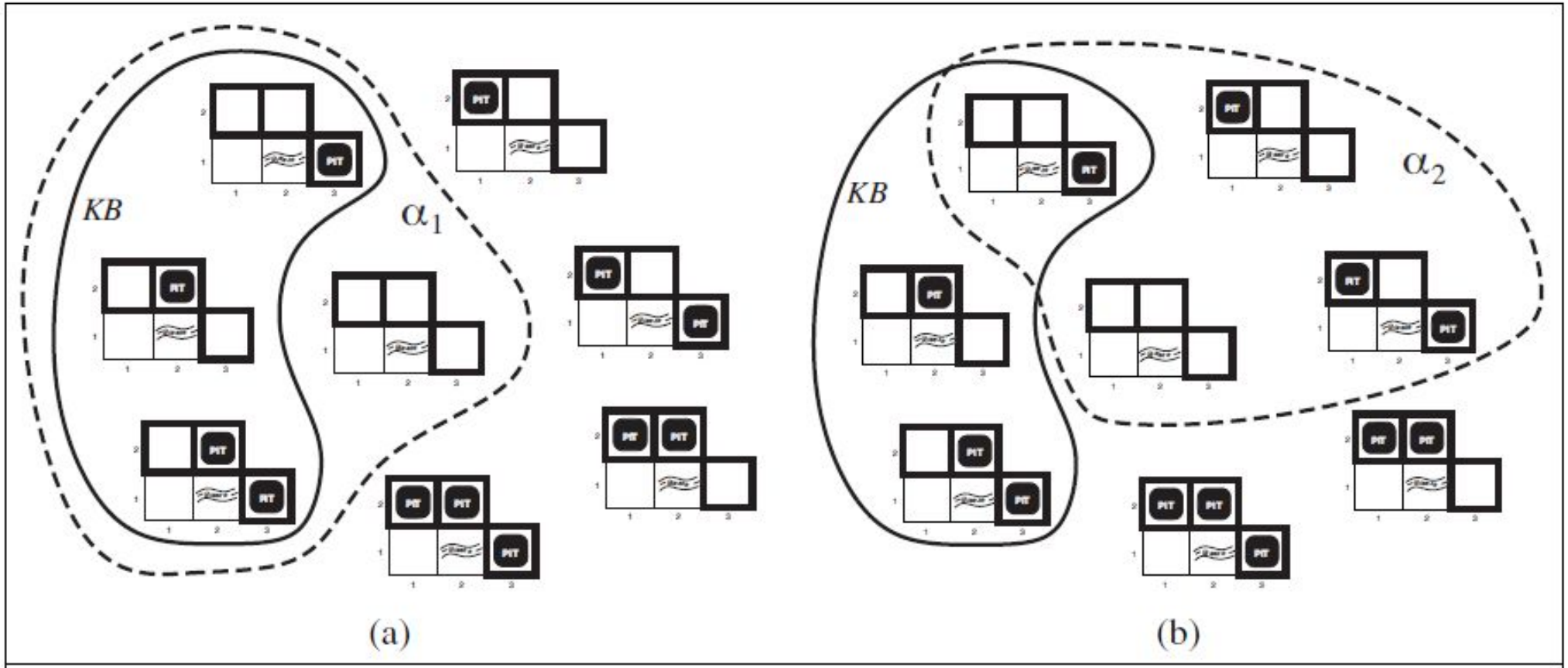


Figure 7.5 Possible models for the presence of pits in squares [1,2], [2,2], and [3,1]. The KB corresponding to the observations of nothing in [1,1] and a breeze in [2,1] is shown by the solid line. (a) Dotted line shows models of α_1 (no pit in [1,2]). (b) Dotted line shows models of α_2 (no pit in [2,2]).

- In the Wumpus World example (Figure 7.3b), the agent starts at [1,1] with no breeze and detects a breeze at [2,1]. This information, plus the rules of the game, forms the knowledge base (KB).
- The agent wants to know if [1,2], [2,2], and [3,1] have pits. Each square may or may not have a pit, so there are $2^3 = 8$ possible models (worlds). Only 3 models match the KB(solid lines in Figure 7.5).
- Two statements are tested:
  - α_1: "No pit in [1,2]"
  - α_2: "No pit in [2,2]"
- From the models:
- α_1 is true in all models where KB is true → KB entails α_1.
- α_2 is not always true when KB is true → KB does not entail α_2.
- This shows entailment (what logically follows from KB) and inference (the process of finding it).

- Model checking tests entailment by listing all possible models and checking if α is true in all models where KB is true.
- Soundness: An inference method is sound if it only produces correct conclusions (model checking is sound).
- Completeness: A method is complete if it can find every conclusion that logically follows from KB.

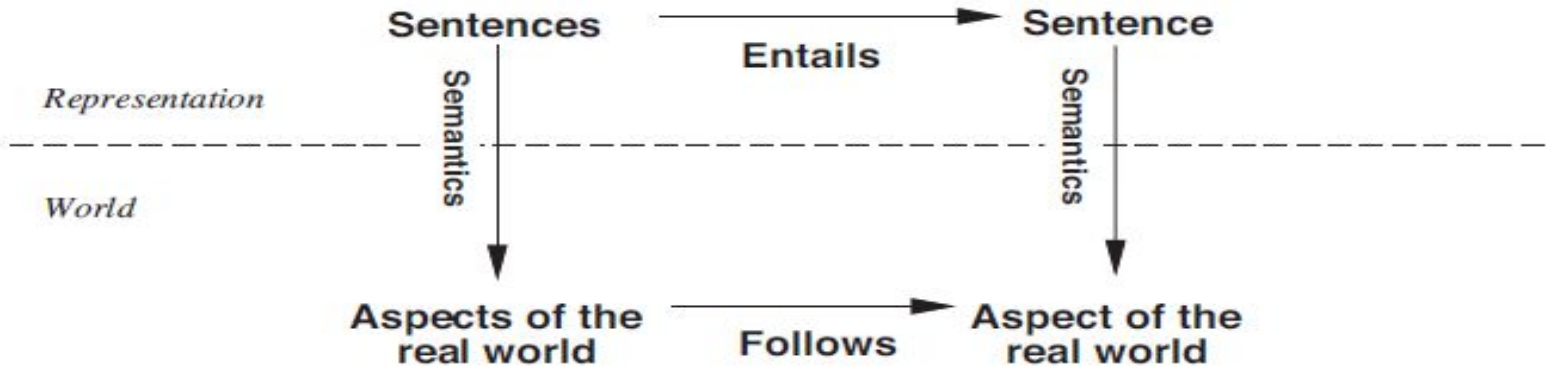


Figure 7.6 Sentences are physical configurations of the agent, and reasoning is a process of constructing new physical configurations from old ones. Logical reasoning should ensure that the new configurations represent aspects of the world that actually follow from the aspects that the old configurations represent.

- The final issue to consider is grounding—the connection between logical reasoning processes and the real environment in which the agent exists. In particular, how do we know that KB is true in the real world? (After all, KB is just "syntax" inside the agent's head.)

- An agent's sensors connect its knowledge to the real world. For example, in the Wumpus World, the agent's smell sensor detects odors and adds a "smell" statement to its knowledge base.
- This means that whenever the sensor detects a smell, the knowledge base reflects something true in the real world. However, not all knowledge comes directly from sensors. Some, like "wumpuses cause smells in nearby squares," are general rules learned from experience.
- Learning isn't perfect—there could be exceptions (like wumpuses bathing on February 29). So, the knowledge base might sometimes be wrong, but with good learning methods, it can still be mostly accurate.

3.3 Propositional Logic

- A very simple but powerful logic called Propositional Logic.

3.3.1 Syntax

- The syntax of propositional logic defines the allowable sentences. The atomic sentences consist of a single proposition symbol. Each such symbol stands for a proposition that can be true or false. We use symbols that start with an uppercase letter and may contain other letters or subscripts,
- for example: P, Q, R, $W_{1,3}$ and North.
- $W_{1,3}$- Wumpus is in [1,3].
- There are two proposition symbols with fixed meanings: True is the always-true proposition and False is the always-false proposition.

- Complex sentences are constructed from simpler sentences, using parentheses and logical connectives. There are five connectives in common use:
- ¬(not). A sentence such as $\neg W_{1,3}$ is called the negation of $W_{1,3}$. A literal is either an atomic sentence (a positive literal) or a negated atomic sentence (a negative literal).
- $\wedge$ (and). A sentence whose main connective is $\wedge$, such as $W_{1,3} \wedge P_{3,1}$, is called a conjunction; its parts are the conjuncts. (The $\wedge$ looks like an "A" for "And.")
- $\vee$ (or). A sentence using $\vee$, such as $(W_{1,3} \wedge P_{3,1}) \vee W_{2,2}$, is a disjunction of the disjuncts $(W_{1,3} \wedge P_{3,1})$ and $W_{2,2}$.

- $\Rightarrow$ (implies). A sentence such as $(W_{1,3} \wedge P_{3,1}) \Rightarrow \neg W_{2,2}$ is called an implication (or conditional). Its premise or antecedent is $(W_{1,3} \wedge P_{3,1})$, and its conclusion or consequent is $\neg W_{2,2}$. Implications are also known as rules or if–then statements. The implication symbol is sometimes written in other books as $\supset$ or $\rightarrow$.
- $\Leftrightarrow$ (if and only if). The sentence $W_{1,3} \Leftrightarrow \neg W_{2,2}$ is a biconditional. Some other books write this as $\equiv$.
- A sentence with several operators can be parsed the grammar in multiple ways. To eliminate the ambiguity we define a precedence for each operator. The "not" operator ($\neg$) has the highest precedence, which means that in the sentence $\neg A \wedge B$ the $\neg$ binds most tightly, giving us the equivalent of $(\neg A) \wedge B$ rather than $\neg (A \wedge B)$.

- $\Rightarrow$ (implies). A sentence such as $(W_{1,3} \wedge P_{3,1}) \Rightarrow \neg W_{2,2}$ is called an implication (or conditional). Its premise or antecedent is $(W_{1,3} \wedge P_{3,1})$, and its conclusion or consequent is $\neg W_{2,2}$. Implications are also known as rules or if–then statements. The implication symbol is sometimes written in other books as $\supset$ or $\rightarrow$.
- $\Leftrightarrow$ (if and only if). The sentence $W_{1,3} \Leftrightarrow \neg W_{2,2}$ is a biconditional. Some other books write this as $\equiv$.
- A sentence with several operators can be parsed the grammar in multiple ways. To eliminate the ambiguity we define a precedence for each operator. The "not" operator ($\neg$) has the highest precedence, which means that in the sentence $\neg A \wedge B$ the $\neg$ binds most tightly, giving us the equivalent of $(\neg A) \wedge B$ rather than $\neg (A \wedge B)$.

$$
\begin{aligned}
\mathit{Sentence} &\rightarrow \mathit{AtomicSentence} \mid \mathit{ComplexSentence} \\
\mathit{AtomicSentence} &\rightarrow \mathit{True} \mid \mathit{False} \mid P \mid Q \mid R \mid \ldots \\
\mathit{ComplexSentence} &\rightarrow (\ \mathit{Sentence}\ ) \mid [\ \mathit{Sentence}\ ] \\
&\mid \neg\ \mathit{Sentence} \\
&\mid \mathit{Sentence} \wedge \mathit{Sentence} \\
&\mid \mathit{Sentence} \vee \mathit{Sentence} \\
&\mid \mathit{Sentence} \Rightarrow \mathit{Sentence} \\
&\mid \mathit{Sentence} \Leftrightarrow \mathit{Sentence} \\
\text{OPERATOR PRECEDENCE} &: \neg, \wedge, \vee, \Rightarrow, \Leftrightarrow
\end{aligned}
$$

Figure 7.7 A BNF (Backus–Naur Form) grammar of sentences in propositional logic, along with operator precedences, from highest to lowest.

3.3.2 Semantics

- The semantics defines the rules for determining the truth of a sentence with respect to a particular model.
- In propositional logic, a model simply fixes the truth value—true or false—for every proposition symbol.
- For example, if the sentences in the knowledge base make use of the proposition symbols $P_{1,2}$, $P_{2,2}$, and $P_{3,1}$, then one possible model is

$$m_1 = \{P_{1,2} = \text{false}, P_{2,2} = \text{false}, P_{3,1} = \text{true}\}$$

With three proposition symbols, there are $2^3 = 8$ possible models—exactly those depicted.

- Notice, however, that the models are purely mathematical objects with no necessary connection to wumpus worlds. $P_{1,2}$ is just a symbol; it might mean "there is a pit in [1,2]" or "I'm in Paris today and tomorrow."
- In propositional logic, semantics define how to determine a sentence's truth value from a model, using recursion. Truth values are first assigned to atomic sentences, then computed for complex sentences using the five connectives.
  - True is true in every model and False is false in every model.
  - The truth value of every other proposition symbol must be specified directly in the model. For example, in the model m1 given earlier, $P_{1,2}$ is false.

- For complex sentences, we have five rules, which hold for any subsentences P and Q in any model m (here "iff" means "if and only if"):
  - ¬P is true iff P is false in m.
  - P $\wedge$ Q is true iff both P and Q are true in m.
  - P $\vee$ Q is true iff either P or Q is true in m.
  - P $\Rightarrow$ Q is true unless P is true and Q is false in m.
  - P $\Leftrightarrow$ Q is true iff P and Q are both true or both false in m
- The rules can also be expressed with truth tables that specify the truth value of a complex sentence for each possible assignment of truth values to its components

P	Q	$\neg P$	$P \wedge Q$	$P \vee Q$	$P \Rightarrow Q$	$P \Leftrightarrow Q$
false	*false*	*true*	*false*	*false*	*true*	*true*
false	*true*	*true*	*false*	*true*	*true*	*false*
true	*false*	*false*	*false*	*true*	*false*	*false*
true	*true*	*false*	*true*	*true*	*true*	*true*

Figure 7.8 Truth tables for the five logical connectives. To use the table to compute, for example, the value of $P \vee Q$ when P is true and Q is false, first look on the left for the row where P is *true* and Q is *false* (the third row). Then look in that row under the $P \vee Q$ column to see the result: *true*.

- The main point of possible confusion is that P $\vee$ Q is true when P is true or Q is true or both. A different connective, called "exclusive or" ("xor" for short), yields false when both disjuncts are true.

In propositional logic, “P ⇒ Q” does not imply causation or relevance—it’s true whenever P is false or both P and Q are true. The only false case is when P is true and Q is false. The biconditional “P ⇔ Q” means both “P ⇒ Q” and “Q ⇒ P” are true (“if and only if”).

Example: $B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})$

states a square is breezy exactly when a neighboring square has a pit.

3.3.3 A simple knowledge base

We focus first on the immutable aspects of the wumpus world, leaving the mutable aspects for a later section. For now, we need the following symbols for each [x, y] location:

$P_{x,y}$ is true if there is a pit in [x, y].

$W_{x,y}$ is true if there is a wumpus in [x, y], dead or alive.

$B_{x,y}$ is true if the agent perceives a breeze in [x, y].

$S_{x,y}$ is true if the agent perceives a stench in [x, y].

The sentences we write will suffice to derive $\neg P_{1,2}$ (there is no pit in [1,2]), as was done informally. We label each sentence R_i so that we can refer to them:

- There is no pit in [1,1]:

$$R_1 : \neg P_{1,1}$$

A square is breezy if and only if there is a pit in a neighboring square. This has to be stated for each square; for now, we include just the relevant squares:

$$R_2 : B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})$$

$$R_3 : B_{2,1} \Leftrightarrow (P_{1,1} \vee P_{2,2} \vee P_{3,1})$$

The preceding sentences are true in all wumpus worlds. Now we include the breeze percepts for the first two squares visited in the specific world the agent is in, leading up to the situation in Figure 7.3(b).

$R_4 : \neg B_{1,1}$

$R_5 : B_{2,1}$

3.3.4 A simple inference procedure

Our first algorithm for inference is a model-checking approach that is a direct implementation of the definition of entailment: enumerate the models, and check that α is true in every model in which KB is true.

Models are assignments of true or false to every proposition symbol. Returning to our wumpus-world example, the relevant proposition symbols are $B_{1,1}$, $B_{2,1}$, $P_{1,1}$, $P_{1,2}$, $P_{2,1}$, $P_{2,2}$, and $P_{3,1}$. With seven symbols, there are $2^7 = 128$ possible models; in three of these, KB is true (Figure 7.9).

In those three models, $\neg P_{1,2}$ is true, hence there is no pit in [1,2]. On the other hand, $P_{2,2}$ is true in two of the three models and false in one, so we cannot yet tell whether there is a pit in [2,2].

$B_{1,1}$	$B_{2,1}$	$P_{1,1}$	$P_{1,2}$	$P_{2,1}$	$P_{2,2}$	$P_{3,1}$	R_1	R_2	R_3	R_4	R_5	KB
false	*false*	*false*	*false*	*false*	*false*	*false*	*true*	*true*	*true*	*true*	*false*	*false*
false	*false*	*false*	*false*	*false*	*false*	*true*	*true*	*true*	*false*	*true*	*false*	*false*
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮
false	*true*	*false*	*false*	*false*	*false*	*false*	*true*	*true*	*false*	*true*	*true*	*false*
false	*true*	*false*	*false*	*false*	*false*	*true*	*true*	*true*	*true*	*true*	*true*	*true*
false	*true*	*false*	*false*	*false*	*true*	*false*	*true*	*true*	*true*	*true*	*true*	*true*
false	*true*	*false*	*false*	*false*	*true*	*true*	*true*	*true*	*true*	*true*	*true*	*true*
false	*true*	*false*	*false*	*true*	*false*	*false*	*true*	*false*	*false*	*true*	*true*	*false*
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮
true	*true*	*true*	*true*	*true*	*true*	*true*	*false*	*true*	*true*	*false*	*true*	*false*

Figure 7.9 A truth table constructed for the knowledge base given in the text. *KB* is true if R_1 through R_5 are true, which occurs in just 3 of the 128 rows (the ones underlined in the right-hand column). In all 3 rows, $P_{1,2}$ is false, so there is no pit in [1,2]. On the other hand, there might (or might not) be a pit in [2,2].

```
function TT-ENTAILS?(KB, α) returns true or false
  inputs: KB, the knowledge base, a sentence in propositional logic
          α, the query, a sentence in propositional logic

  symbols ← a list of the proposition symbols in KB and α
  return TT-CHECK-ALL(KB, α, symbols, { })
```

```
function TT-CHECK-ALL(KB, α, symbols, model) returns true or false
  if EMPTY?(symbols) then
      if PL-TRUE?(KB, model) then return PL-TRUE?(α, model)
      else return true // when KB is false, always return true
  else do
      P ← FIRST(symbols)
      rest ← REST(symbols)
      return (TT-CHECK-ALL(KB, α, rest, model ∪ {P = true})
              and
              TT-CHECK-ALL(KB, α, rest, model ∪ {P = false }))
```

Figure 7.10 A truth-table enumeration algorithm for deciding propositional entailment. (TT stands for truth table.) PL-TRUE? returns *true* if a sentence holds within a model. The variable *model* represents a partial model—an assignment to some of the symbols. The keyword "**and**" is used here as a logical operation on its two arguments, returning *true* or *false*.

$$
\begin{aligned}
(\alpha \wedge \beta) &\equiv (\beta \wedge \alpha) && \text{commutativity of } \wedge \\
(\alpha \vee \beta) &\equiv (\beta \vee \alpha) && \text{commutativity of } \vee \\
((\alpha \wedge \beta) \wedge \gamma) &\equiv (\alpha \wedge (\beta \wedge \gamma)) && \text{associativity of } \wedge \\
((\alpha \vee \beta) \vee \gamma) &\equiv (\alpha \vee (\beta \vee \gamma)) && \text{associativity of } \vee \\
\neg(\neg\alpha) &\equiv \alpha && \text{double-negation elimination} \\
(\alpha \Rightarrow \beta) &\equiv (\neg\beta \Rightarrow \neg\alpha) && \text{contraposition} \\
(\alpha \Rightarrow \beta) &\equiv (\neg\alpha \vee \beta) && \text{implication elimination} \\
(\alpha \Leftrightarrow \beta) &\equiv ((\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)) && \text{biconditional elimination} \\
\neg(\alpha \wedge \beta) &\equiv (\neg\alpha \vee \neg\beta) && \text{De Morgan} \\
\neg(\alpha \vee \beta) &\equiv (\neg\alpha \wedge \neg\beta) && \text{De Morgan} \\
(\alpha \wedge (\beta \vee \gamma)) &\equiv ((\alpha \wedge \beta) \vee (\alpha \wedge \gamma)) && \text{distributivity of } \wedge \text{ over } \vee \\
(\alpha \vee (\beta \wedge \gamma)) &\equiv ((\alpha \vee \beta) \wedge (\alpha \vee \gamma)) && \text{distributivity of } \vee \text{ over } \wedge
\end{aligned}
$$

Figure 7.11 Standard logical equivalences. The symbols α, β, and γ stand for arbitrary sentences of propositional logic.